Horizon Capital

Implementation Guide & Test Cases

v0.2.0 | 2026-03-24 | 300+ tests passing

Part 1	**Implementation Steps**	Phases 1-9 with commands to verify each
Part 2	**Test Cases T-01 to T-20**	20 test groups, 300+ individual assertions
Part 3	**Quick Reference**	Run commands, lint, coverage gaps

9	300+	40+	23	26+	4
Phases	Tests	REPL Commands	DB Tables	Backtest Metrics	ML Models

Horizon Capital -- Implementation Guide & Test Cases

Version 0.2.0 · 2026-03-24

Part 1: Implementation Steps

Phase 1 -- Core Infrastructure [DONE]

Steps completed:

- Define 4-department agent hierarchy: Front Office, Middle Office, Back Office, Executive
- Implement BaseAgent with start/stop/heartbeat lifecycle
- Build AgentBus -- in-memory (asyncio.Queue) for LOCAL, Redis pub/sub for POSTGRES
- Create LocalStorage -- 23-table SQLite schema via SQLAlchemy 2.x async
- Wire Runtime -- get_bus(), get_session_factory(), lazy Redis/asyncpg imports
- Add Settings via Pydantic with .env file support
- Build Orchestrator -- starts all agents, monitors heartbeats, auto-restarts crashed workers (3 retries)
- Add CLI entry point (__main__.py) with run, status, config, version subcommands

Verify:

```
python -m pytest tests/test_smoke.py::TestConfig -v
python -m pytest tests/test_smoke.py::TestLocalStorage -v
python -m pytest tests/test_smoke.py::TestMemoryBus -v
python -m pytest tests/test_integration.py::TestOrchestratorLifecycle -v
```

Phase 2 -- Data Layer [DONE]

Steps completed:

- Define canonical schemas: NormalizedTick, NormalizedOHLCV (Pydantic)
- Implement YahooFeed -- 5-min poll, symbol suffix mapping (.NS, .BO, etc.)
- Implement AlpacaFeed -- WebSocket ticks + REST fallback every 10s
- Implement PolygonFeed -- 60s poll, 5-min bars, adjusted=true
- Implement CoinGeckoFeed -- 60s poll, 10 coins, synthetic OHLCV from spot price
- Implement ZerodhaFeed -- mock random-walk ticks at 2s (real KiteTicker TODO)
- Implement ForexFeed -- USD/INR from Yahoo USDINR=X, cached in Redis
- Build DataIngestionWorker -- polls all feeds, circuit breaker on each, writes to ohlcv + ticks tables
- Redis tick cache: horizon:tick:{symbol}, TTL 5 min

Verify:

```
python -m pytest tests/test_integration.py::TestTechnicalIndicators -v
```

Phase 3 -- Feature Engineering [DONE]

Steps completed:

- Implement 23 technical indicators via ta library in features/technical.py
- Implement VolatilityFeatures: realized vol (20d), GJR-GARCH(1,1) forecast, VIX proxy
- Implement RegimeFeatures: 6D vector (returns, 20d vol, 60d vol, vol-of-vol, skew, kurtosis)
- Build FeatureStore -- write to PostgreSQL features JSONB table + Redis cache (TTL 1h)
- Build FeatureComputationWorker -- runs every 5 min per symbol, publishes FeatureComputed on bus

Verify:

```
python -m pytest tests/test_integration.py::TestTechnicalIndicators -v
python -m pytest tests/test_smoke.py::TestGarchVolatility -v
python -m pytest tests/test_smoke.py::TestOUMeanReversion -v
```

Phase 4 -- ML Models & Regime Detection [DONE]

Steps completed:

- Implement GaussianHMM wrapper (4 states: BULL, BEAR, HIGH_VOL, CRISIS) via hmmlearn
- Build RegimeAgent -- trains HMM, publishes regime + confidence every 5 min
- Implement LFRDriftDetector -- tracks TPR/TNR/PPV/NPV with EWMA (beta=0.9)
- Implement FinBERTScorer (70%) + LoughranMcDonaldDictionary (30%) ensemble
- Build SentimentAgent -- aggregates news + Reddit, weights by log(1 + engagement)
- Build ModelRegistry -- version models, Thompson Sampling A/B test, promote better models

Verify:

```
python -m pytest tests/test_smoke.py::TestJumpModel -v
python -m pytest tests/test_smoke.py::TestLoughranMcDonald -v
python -m pytest tests/test_smoke.py::TestMovedAgentImports -v
```

Phase 5 -- Strategies, Debate & Risk [DONE]

Steps completed:

- Implement MomentumStrategy -- RSI + MACD + VPVMA + EMA + Volume + ATR fusion (BULL regime)
- Implement MeanReversionStrategy -- OU s-score (threshold 1.25) with Bollinger fallback (BEAR/NEUTRAL)
- Implement VolatilityStrategy -- GJR-GARCH vol spike detection (HIGH_VOL regime)
- Implement CrisisStrategy -- trend-following overlay + drawdown-based hedging (CRISIS regime)
- Build EnsembleSelector -- routes regime to appropriate strategy
- Build DebateEngine -- 4-role LLM debate (Strategy, Risk, Sentiment, Portfolio)
- Implement VaRCalculator -- GARCH-EVT (GPD tail fit) with historical simulation fallback
- Implement KillSwitch -- liquidate all + disable agents on threshold breach
- Implement PortfolioOptimizer -- Half-Kelly + Ledoit-Wolf shrinkage
- Build AlmgrenChrissModel -- market impact for large orders
- Build ExecutionAgent -- paper order routing with mock fills

Verify:

```
python -m pytest tests/test_smoke.py::TestEnsembleSelector -v
python -m pytest tests/test_smoke.py::TestVaRCalculator -v
python -m pytest tests/test_smoke.py::TestGarchEvtVaR -v
python -m pytest tests/test_smoke.py::TestHRPPortfolio -v
python -m pytest tests/test_smoke.py::TestVPVMAMomentum -v
python -m pytest tests/test_smoke.py::TestCrisisTrendFollowing -v
python -m pytest tests/test_smoke.py::TestAlmgrenChriss -v
```

Phase 6 -- Advanced Services [DONE]

Steps completed:

- Build StressTesting -- 10+ historical crisis scenarios (GFC 2008, COVID 2020, etc.)
- Build AlertManager -- rule-based alerts (VaR breach, drawdown, drift, broker disconnect)
- Build ModelRegistry -- Thompson Sampling for strategy A/B testing

- Build ReportGenerator -- daily/weekly/monthly Rich + exportable reports
- Build HealthMonitor -- agent heartbeats, DB health, feed status, LLM availability

Verify:

```
python -m pytest tests/test_integration.py::TestStressTesting -v
python -m pytest tests/test_smoke.py::TestAdvancedMetrics -v
```

Phase 7 -- Production Hardening [DONE]

Steps completed:

- Implement CircuitBreaker (app/core/resilience.py) -- CLOSED/OPEN/HALF_OPEN states
- Add retry_with_backoff -- exponential backoff, max retries configurable
- Add with_timeout -- async timeout wrapper
- Add PerformanceWorker -- daily Sharpe, Sortino, Calmar, alpha vs NIFTY 50 / S&P 500
- Add AlphaMonitor -- decay tracking, rolling Sharpe, per-strategy attribution
- Add AdaptationEngine -- dynamic strategy weight adjustments based on regime performance
- Implement security: password validation, JWT, CORS from env, security headers
- Pin all dependencies with version ranges in pyproject.toml
- Ruff linting: 0 errors across entire app/ directory

Verify:

```
python -m pytest tests/test_integration.py::TestCircuitBreaker -v
python -m pytest tests/test_integration.py::TestRetryWithBackoff -v
python -m pytest tests/test_integration.py::TestTimeout -v
python -m pytest tests/test_integration.py::TestSecurity -v
python -m pytest tests/test_integration.py::TestPerformanceService -v
python -m pytest tests/test_smoke.py::TestAlphaMonitor -v
python -m pytest tests/test_smoke.py::TestAdaptationEngine -v
```

Phase 8 -- Backtesting Engine [DONE]

Steps completed:

- Build event-driven MultiAssetBacktestEngine -- processes bars in chronological order
- Implement walk-forward splits (in-sample / out-of-sample windows)
- Add 26+ backtest metrics: Sharpe, Sortino, Calmar, Omega ratio, tail ratio, recovery factor, information ratio, avg win/loss, max consecutive losses, drawdown duration
- Add per-strategy and per-symbol performance attribution
- Wire backtest REPL command with --days, --strategy flags
- Add backtest results command for last-run summary

Verify:

```
python -m pytest tests/test_smoke.py::TestMultiAssetEngine -v
python -m pytest tests/test_smoke.py::TestAdvancedMetrics -v
python -m pytest tests/test_integration.py::TestCommandHandler -k "backtest" -v
```

Phase 9 -- Remaining TODOs [PENDING]

Item	File	Effort	Priority
Zerodha KiteTicker WebSocket	data_layer/zerodha_feed.py	Small	Medium
Binance API for crypto execution	new binance_feed.py	Medium	Medium
HRP portfolio optimizer (live)	agents/front_office/portfolio/optimizer.py	Small	Low

Continuous HMM retraining pipeline	workers/	Medium	High
Optuna hyperparameter tuning	ml_training/	Medium	Low
SHAP feature importance	features/	Small	Low
Multi-currency P&L (INR + USD)	agents/back_office/accounting/	Medium	High
SEBI AIF Cat III compliance checks	agents/middle_office/compliance/	Large	High

Part 2: Test Cases

Full Test Run

```
# Run all 300+ tests
python -m pytest tests/ -v

# With coverage
python -m pytest tests/ --cov=app --cov-report=term-missing -v

# Fast smoke only (no integration)
python -m pytest tests/test_smoke.py -v

# Integration only
python -m pytest tests/test_integration.py -v
```

T-01: Configuration Loading

File: test_smoke.py::TestConfig

What it tests: Settings load from env, symbol lists parse, CIO mode flags

```
# test_settings_load
settings = get_settings()
assert settings.app_name == "Horizon Capital"
assert settings.storage_backend in ("LOCAL", "POSTGRES")

# test_trading_symbols_parsed
assert len(settings.india_symbols) > 0
assert "RELIANCE.NSE" in settings.india_symbols

# test_cio_mode
assert isinstance(settings.cio_mode, str)
```

Run: pytest tests/test_smoke.py::TestConfig -v

Expected: 4/4 pass

T-02: Local Storage (SQLite)

File: test_smoke.py::TestLocalStorage

What it tests: SQLite schema creates all 23 tables; insert + query round-trip works

```
# test_local_storage_init
storage = LocalStorage(":memory:")
await storage.initialize()
# Verifies all 23 tables exist

# test_insert_and_query
await storage.insert_ohlcv(symbol="AAPL", open=150.0, high=155.0,
                           low=149.0, close=152.0, volume=1000000)
rows = await storage.query_ohlcv("AAPL", limit=1)
assert rows[0]["close"] == 152.0
```

Run: pytest tests/test_smoke.py::TestLocalStorage -v

Expected: 2/2 pass

T-03: In-Memory Message Bus

File: test_smoke.py::TestMemoryBus

What it tests: Publish/subscribe routing; wildcard topic subscriptions

```
# test_pub_sub
received = []
bus.subscribe("signals.*", lambda msg: received.append(msg))
await bus.publish(AgentMessage(topic="signals.AAPL", payload={"dir": 1}))
await asyncio.sleep(0.01)
assert len(received) == 1

# test_wildcard
bus.subscribe("*", lambda msg: received.append(msg))
# Should receive all messages
```

Run: pytest tests/test_smoke.py::TestMemoryBus -v

Expected: 2/2 pass

T-04: Circuit Breaker

File: test_integration.py::TestCircuitBreaker

What it tests: State machine transitions CLOSED -> OPEN -> HALF_OPEN -> CLOSED

```
# test_opens_after_threshold
cb = CircuitBreaker(failure_threshold=3, recovery_timeout=0.1)
for _ in range(3):
    with pytest.raises(Exception):
        async with cb:
            raise ValueError("feed error")
assert cb.state == "OPEN"

# test_half_open_recovery
await asyncio.sleep(0.15)          # wait past recovery_timeout
async with cb:
    pass                           # success in HALF_OPEN
assert cb.state == "CLOSED"

# test_reset
cb.reset()
assert cb.state == "CLOSED"
assert cb.failure_count == 0
```

Run: pytest tests/test_integration.py::TestCircuitBreaker -v

Expected: 4/4 pass

T-05: Retry with Backoff

File: test_integration.py::TestRetryWithBackoff

What it tests: Eventual success after transient failures; raises after max retries

```
# test_succeeds_on_retry
call_count = 0
@retry_with_backoff(max_retries=3, base_delay=0.01)
async def flaky():
    nonlocal call_count
    call_count += 1
    if call_count < 3:
        raise ConnectionError("transient")
    return "ok"

result = await flaky()
assert result == "ok"
assert call_count == 3

# test_exhausts_retries
with pytest.raises(ConnectionError):
    await always_fails()   # raises every time
```

Run: pytest tests/test_integration.py::TestRetryWithBackoff -v

Expected: 2/2 pass

T-06: Technical Indicators

File: test_integration.py::TestTechnicalIndicators

What it tests: Correctness of SMA, RSI bounds, MACD sign, Bollinger bandwidth, GARCH output

```
# test_sma
prices = pd.Series([10.0] * 25)
sma = compute_sma(prices, window=20)
assert sma.iloc[-1] == pytest.approx(10.0)

# test_rsi_bounds
rsi = compute_rsi(random_prices, window=14)
assert rsi.dropna().between(0, 100).all()

# test_macd
macd, signal, hist = compute_macd(trending_prices)
assert (hist > 0).any() or (hist < 0).any()   # has signal

# test_bollinger_bands
upper, mid, lower = compute_bollinger(prices, window=20, std_dev=2.0)
assert (upper >= mid).all()
assert (mid >= lower).all()

# test_volatility_features
result = compute_volatility_features(prices)
assert "realized_vol" in result
assert "garch_forecast" in result
assert result["realized_vol"] >= 0
```

Run: pytest tests/test_integration.py::TestTechnicalIndicators -v

Expected: 6/6 pass

T-07: VaR Calculator

File: test_smoke.py::TestVaRCalculator + TestGarchEvtVaR

What it tests: Empty data returns zero; normal distribution; max/current drawdown; GARCH-EVT path

```
# test_empty_var_returns_zero
calc = VaRCalculator()
result = calc.calculate(returns=pd.Series([], dtype=float))
assert result["var_95"] == 0.0

# test_normal_distribution_var
returns = pd.Series(np.random.normal(0, 0.01, 500))
result = calc.calculate(returns)
assert result["var_95"] > 0
assert result["var_99"] > result["var_95"]   # 99% > 95%

# test_max_drawdown
equity = pd.Series([100, 110, 90, 95, 105])
dd = calc.max_drawdown(equity)
assert dd == pytest.approx(0.1818, rel=0.01)   # (110-90)/110

# test_garch_evt_path
returns = pd.Series(np.random.normal(0, 0.01, 200))
result = calc.calculate_garch_evt(returns, confidence=0.99)
assert "var" in result
assert "cvar" in result
assert result["cvar"] >= result["var"]
```

Run: pytest tests/test_smoke.py::TestVaRCalculator tests/test_smoke.py::TestGarchEvtVaR -v

Expected: 8/8 pass

T-08: HRP Portfolio Optimizer

File: test_smoke.py::TestHRPPortfolio

What it tests: Weights sum to 1, Ledoit-Wolf positive-definite, Kelly fraction bounds, position sizing

```
# test_hrp_weights_sum_to_one
returns_matrix = pd.DataFrame(np.random.randn(252, 5) * 0.01,
                              columns=["A","B","C","D","E"])
weights = hrp_optimizer.optimize(returns_matrix)
assert sum(weights.values()) == pytest.approx(1.0, abs=0.001)
assert all(w >= 0 for w in weights.values())

# test_ledoit_wolf_positive_definite
cov = ledoit_wolf_shrinkage(returns_matrix)
eigenvalues = np.linalg.eigvalsh(cov)
assert all(eigenvalues > 0)         # positive definite

# test_kelly_fraction
f = kelly_fraction(win_rate=0.55, avg_win=0.02, avg_loss=0.01)
assert 0 < f <= 1.0                 # valid fraction

# test_half_kelly_position_sizing
size = half_kelly_position(portfolio_value=100_000,
                           win_rate=0.55, avg_win=0.02, avg_loss=0.01)
assert size <= 50_000               # never more than 50% of portfolio
```

Run: pytest tests/test_smoke.py::TestHRPPortfolio -v

Expected: 4/4 pass

T-09: Sentiment Analysis

File: test_smoke.py::TestLoughranMcDonald

What it tests: Positive/negative/neutral classification, uncertainty and litigious detection

```
# test_positive_text
scorer = LoughranMcDonaldDictionary()
result = scorer.score("Strong revenue growth exceeded expectations profit surged")
assert result["sentiment"] == "positive"
assert result["score"] > 0

# test_negative_text
result = scorer.score("Bankruptcy risk failure loss decline crisis")
assert result["sentiment"] == "negative"
assert result["score"] < 0

# test_neutral_text
result = scorer.score("The company filed its quarterly report on Monday")
assert result["sentiment"] == "neutral"

# test_uncertainty_detection
result = scorer.score("The outcome remains unclear and uncertain perhaps")
assert result["uncertainty_count"] > 0

# test_litigious_detection
result = scorer.score("The defendant faces litigation lawsuit verdict")
assert result["litigious_count"] > 0
```

Run: pytest tests/test_smoke.py::TestLoughranMcDonald -v

Expected: 5/5 pass

T-10: VPVMA Momentum Strategy

File: test_smoke.py::TestVPVMAMomentum

What it tests: VPVMA computation, fusion scoring, signal direction on trending data

```
# test_vpvma_computation
prices = pd.Series(np.linspace(100, 120, 50))   # uptrend
volumes = pd.Series([1_000_000] * 50)
vpvma = compute_vpvma(prices, volumes)
assert vpvma is not None
assert len(vpvma) == len(prices)

# test_fusion_score_bounds
features = build_features(prices, volumes)
score = momentum_fusion_score(features)
assert -1.0 <= score <= 1.0

# test_buy_signal_on_uptrend
signal = MomentumStrategy().generate_signal("AAPL.NASDAQ", features, regime="BULL")
assert signal is not None
assert signal.direction == 1        # BUY
assert signal.strength > 0.5
```

Run: pytest tests/test_smoke.py::TestVPVMAMomentum -v

Expected: 3/3 pass

T-11: OU Mean Reversion

File: test_smoke.py::TestOUMeanReversion

What it tests: OU calibration on mean-reverting series, trending detection, Bollinger fallback

```
# test_ou_calibration_mean_reverting
# Simulate OU process
theta, kappa, sigma, s_score = calibrate_ou(mean_reverting_prices)
assert kappa > 0                    # positive mean reversion speed
assert abs(s_score) >= 0            # s-score computed

# test_ou_calibration_trending
# Linear trend should have low kappa (not mean-reverting)
theta, kappa, sigma, s_score = calibrate_ou(trending_prices)
# Low kappa means OU model doesn't apply; fallback kicks in

# test_bollinger_fallback_when_insufficient_data
signal = MeanReversionStrategy().generate_signal(
    "RELIANCE.NSE", features_with_few_samples, regime="BEAR")
assert signal.meta.get("method") == "bollinger_fallback"
```

Run: pytest tests/test_smoke.py::TestOUMeanReversion -v

Expected: 3/3 pass

T-12: Crisis Trend-Following Strategy

File: test_smoke.py::TestCrisisTrendFollowing

What it tests: Trend detection, drawdown-triggered hedging, VIX-based scaling, trailing stop

```
# test_trend_following_in_crisis
features = {"ema_10": 95.0, "ema_50": 100.0, "atr_14": 2.0}  # downtrend
signal = CrisisStrategy().generate_signal("AAPL", features, regime="CRISIS")
assert signal.direction == -1       # SELL / hedge in downtrend

# test_drawdown_hedging
features["current_drawdown"] = 0.12   # 12% > 5% threshold
signal = CrisisStrategy().generate_signal("AAPL", features, regime="CRISIS")
assert signal.meta["hedge_ratio"] > 0

# test_vix_scaling
features["vix_proxy"] = 35.0       # elevated VIX
signal = CrisisStrategy().generate_signal("AAPL", features, regime="CRISIS")
assert signal.strength > 0.7       # stronger signal at high VIX

# test_trailing_stop_set
assert signal.stop_loss is not None
# Stop loss should be 3x ATR from entry
```

Run: pytest tests/test_smoke.py::TestCrisisTrendFollowing -v

Expected: 5/5 pass

T-13: Almgren-Chriss Market Impact

File: test_smoke.py::TestAlmgrenChriss

What it tests: Impact grows with order size, zero quantity returns zero, impact decomposition

```
# test_market_impact_positive
model = AlmgrenChrissModel(avg_daily_volume=1_000_000, volatility=0.02)
impact = model.estimate_impact(quantity=10_000, side="BUY")
assert impact["total_impact_bps"] > 0

# test_impact_increases_with_size
small = model.estimate_impact(quantity=1_000, side="BUY")
large = model.estimate_impact(quantity=100_000, side="BUY")
assert large["total_impact_bps"] > small["total_impact_bps"]

# test_zero_quantity
impact = model.estimate_impact(quantity=0, side="BUY")
assert impact["total_impact_bps"] == 0

# test_impact_breakdown
assert "permanent_impact_bps" in impact
assert "temporary_impact_bps" in impact
```

Run: pytest tests/test_smoke.py::TestAlmgrenChriss -v

Expected: 5/5 pass

T-14: Performance Metrics

File: test_integration.py::TestPerformanceService

What it tests: Sharpe, Sortino, Calmar, Omega, max drawdown, win rate, alpha

```
# test_sharpe_ratio
returns = pd.Series([0.001, -0.002, 0.003, 0.001, -0.001] * 50)
sharpe = calc_sharpe(returns, risk_free=0.05/252)
assert sharpe == pytest.approx(expected_sharpe, rel=0.05)

# test_sortino_ratio
sortino = calc_sortino(returns)
assert sortino >= sharpe            # Sortino >= Sharpe when returns skewed positive

# test_max_drawdown
equity = pd.Series([1.0, 1.1, 0.9, 0.95, 1.05])
dd = calc_max_drawdown(equity)
assert dd == pytest.approx(0.1818, rel=0.01)

# test_alpha_vs_benchmark
portfolio_returns = pd.Series(np.random.normal(0.001, 0.01, 252))
benchmark_returns = pd.Series(np.random.normal(0.0005, 0.012, 252))
alpha = calc_alpha(portfolio_returns, benchmark_returns, risk_free=0.0)
assert isinstance(alpha, float)    # can be positive or negative

# test_win_rate
trades = [0.02, -0.01, 0.03, 0.01, -0.02]
wr = calc_win_rate(trades)
assert wr == pytest.approx(0.6)    # 3 of 5 positive
```

Run: pytest tests/test_integration.py::TestPerformanceService -v

Expected: 11/11 pass

T-15: Security

File: test_integration.py::TestSecurity

What it tests: Password hashing, validation rules, JWT round-trip, refresh token, production warnings

```
# test_password_hashing
hashed = hash_password("SecurePass1")
assert verify_password("SecurePass1", hashed) is True
assert verify_password("WrongPass", hashed) is False

# test_password_validation_length
assert validate_password("Short1") is False      # < 8 chars
assert validate_password("LongEnough1") is True

# test_password_validation_uppercase
assert validate_password("nouppercase1") is False

# test_password_validation_digit
assert validate_password("NoDigitHere") is False

# test_jwt_roundtrip
token = create_jwt(user_id="admin", expiry_minutes=30)
payload = decode_jwt(token)
assert payload["sub"] == "admin"
assert payload["exp"] > time.time()

# test_refresh_token
refresh = create_refresh_token(user_id="admin")
assert len(refresh) > 32           # sufficiently random
```

Run: pytest tests/test_integration.py::TestSecurity -v

Expected: 8/8 pass (note: bcrypt tests may skip on Python 3.14 due to passlib compatibility)

T-16: REPL Commands

File: test_integration.py::TestCommandHandler

What it tests: 13 REPL commands work end-to-end

```
# test_help_returns_commands
resp = await handler.handle("help")
assert "Available commands" in resp

# test_unknown_command
resp = await handler.handle("zzz-invalid")
assert "Unknown command" in resp

# test_portfolio_command
resp = await handler.handle("portfolio")
assert "NAV" in resp or "Portfolio" in resp

# test_risk_command
resp = await handler.handle("risk")
assert "VaR" in resp or "Drawdown" in resp

# test_mode_change
resp = await handler.handle("mode PAPER_TRADING")
assert "PAPER_TRADING" in resp

# test_risk_tolerance
resp = await handler.handle("risk-tolerance 15")
assert "15" in resp or "updated" in resp.lower()

# test_performance_command
resp = await handler.handle("performance 30d")
assert "Sharpe" in resp or "Return" in resp
```

Run: pytest tests/test_integration.py::TestCommandHandler -v

Expected: 13/13 pass

T-17: Stress Testing

File: test_integration.py::TestStressTesting

What it tests: Scenarios exist, run without error, all produce negative portfolio impact

```
# test_historical_scenarios_exist
svc = StressTestingService()
scenarios = svc.list_scenarios()
names = [s["name"] for s in scenarios]
assert "GFC 2008" in names
assert "COVID 2020" in names
assert "Crypto Winter 2022" in names

# test_run_all_scenarios
portfolio = build_test_portfolio()
results = await svc.run_all(portfolio)
assert len(results) >= 5            # at least 5 scenarios

# test_scenarios_produce_negative_impact
for result in results:
    assert result["portfolio_impact_pct"] < 0    # all crises are bad
    assert "max_drawdown" in result
```

Run: pytest tests/test_integration.py::TestStressTesting -v

Expected: 3/3 pass

T-18: Dashboard Layout

File: test_integration.py::TestDashboard

What it tests: Signal/log/alert limits enforced, layout renders without error

```
# test_signal_limit
dash = Dashboard()
for i in range(120):
    dash.add_signal({"symbol": "AAPL", "direction": 1, "strength": 0.7})
assert len(dash.recent_signals) <= 100    # max 100 signals kept

# test_agent_status_update
dash.update_agent_status("regime_agent", "RUNNING", {"regime": "BULL"})
statuses = dash.get_agent_statuses()
assert statuses["regime_agent"]["status"] == "RUNNING"

# test_regime_update
dash.update_regime("US", "BULL", confidence=0.87)
assert dash.regimes["US"]["regime"] == "BULL"
assert dash.regimes["US"]["confidence"] == pytest.approx(0.87)

# test_layout_renders
layout = dash.build_layout()
assert layout is not None            # Rich Layout object
```

Run: pytest tests/test_integration.py::TestDashboard -v

Expected: 6/6 pass

T-19: Agent Communication

File: test_integration.py::TestAgentCommunication

What it tests: Multi-handler dispatch, signal-to-risk message flow, unsubscribe

```
# test_multi_handler_dispatch
results = []
bus.subscribe("signals.*", lambda m: results.append(("h1", m)))
bus.subscribe("signals.*", lambda m: results.append(("h2", m)))
await bus.publish(AgentMessage(topic="signals.AAPL", payload={}))
await asyncio.sleep(0.01)
assert len(results) == 2            # both handlers fired

# test_signal_to_risk_flow
# Simulate StrategyAgent publishing, RiskAgent receiving
signal_received = asyncio.Event()
bus.subscribe("risk.*", lambda m: signal_received.set())
await bus.publish(AgentMessage(topic="signals.new", payload={"direction": 1}))
await asyncio.wait_for(signal_received.wait(), timeout=1.0)

# test_unsubscribe
sub_id = bus.subscribe("test.*", lambda m: None)
bus.unsubscribe(sub_id)
# Publishing should not reach removed handler
```

Run: pytest tests/test_integration.py::TestAgentCommunication -v

Expected: 3/3 pass

T-20: Orchestrator Lifecycle

File: test_integration.py::TestOrchestratorLifecycle

What it tests: All agents start, departments report status, graceful shutdown

```
# test_start_stop_lifecycle
orch = Orchestrator(runtime)
await orch.start()
await asyncio.sleep(0.5)            # let agents initialize
statuses = orch.get_all_statuses()
assert all(s["status"] in ("RUNNING", "IDLE") for s in statuses.values())
await orch.stop()
assert all(s["status"] == "STOPPED" for s in orch.get_all_statuses().values())

# test_department_statuses
dept_statuses = orch.get_department_statuses()
assert "FRONT_OFFICE" in dept_statuses
assert "MIDDLE_OFFICE" in dept_statuses

# test_worker_registration
assert "data_ingestion" in orch.registered_workers
assert "feature_computation" in orch.registered_workers
assert "performance" in orch.registered_workers
```

Run: pytest tests/test_integration.py::TestOrchestratorLifecycle -v

Expected: 3/3 pass

Part 3: Quick Reference

Run Everything

```
# All tests
python -m pytest tests/ -v --tb=short

# Smoke only (fast, ~10s)
python -m pytest tests/test_smoke.py -v

# Integration only (~30s)
python -m pytest tests/test_integration.py -v

# Single class
python -m pytest tests/test_smoke.py::TestGarchEvtVaR -v

# Keyword filter
python -m pytest tests/ -k "garch or var or risk" -v
```

Lint Check

```
cd backend
ruff check app/       # expect: 0 errors
```

Boot System Locally

```
cd backend
cp .env.example .env  # fill API keys
python -m app run     # TUI mode
python -m app run --headless  # background mode
```

Test Coverage Gaps (Not Yet Covered)

Area	Missing Test	Reason
Alpaca WebSocket reconnect	No mock WebSocket server	Setup complexity
CoinGecko rate limit handling	No HTTP mock	Needs httpx mock
Zerodha real KiteTicker	Not implemented	Pending Phase 9
Debate Engine LLM call	Needs mocked LLM	Cost of live LLM calls
Redis pub/sub (POSTGRES mode)	Needs running Redis	Infrastructure
Full backtest on real data	Slow (minutes)	Excluded from CI

All 300+ tests pass on Python 3.14, Windows 11, LOCAL mode (no Docker required).